1) Signal problem solving

1) Signal 8-bit Addition and subtraction:-

Sol Given

+45, -23

W.k.T

| | |
|---|---|
| 2 | 45 |
| 2 | 22-1 |
| 2 | 11-0 |
| 2 | 5-1 |
| 2 | 2-1 |
| | 1-0 |

45 = 101101

00101101

| | |
|---|---|
| 2 | 23 |
| 2 | 11-1 |
| 2 | 5-1 |
| 2 | 2-1 |
| | 1-0 |

23 = 10111

00010111

-23 00010111

11101000

11101001

Addition:-

00101101

11101001

00010110

=+22

Subtraction:-

45-(-23) = 45+23 = 68

00101101

00010110

01000100 = +68

2) 4 bit carry load Ahead Adder:-

A carry load Ahead Adder (CLA) is cred to Reduce the delay caused by carry propagation in a Pipeline carry Adder.

1. Generate and Propagate signals:-

$$G_i = A_i B_i$$

Propogate signal $P_i = A_i \oplus B_i$

2. Carry Equations:-

$$C_1 = G_0 + P_0 C_0$$

$$C_2 = G_1 + P_1 G_0 + P_1 P_0 C_0$$

$$C_3 = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 G_0$$

$$C_4 = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 G_0$$

$$S_i = P \oplus C_i$$

3, Working of CLA:-

The CLA calculates the carry signals directly using the generate and propagate signals. The carry does not need to wait for the previous full adder stage.

$C_1, C_2, C_3, C_4$.

4. Comparision

| Carry Calculation | Sequential | Parallel |
|---|---|---|
| Speed | Slow | low |
| Propogated delay | High | low |
| Hardware complexity | low | Higher |
| Performance | lower | Higher |

Booth's Algorithm Table:-

| Step | A | Q | $Q_{-1}$ | & $Q_{-1}$ | |
|---|---|---|---|---|---|
| Initial | 00000 | 00101 | 0 | — | — |
| 1 | 00110 | 00101 | 0 | 10 | A-m |
| Shift | 00011 | 00010 | 1 | — | Right + Shift |
| 2 | 11101 | 00010 | 1 | 01 | A+m |
| Shift | 11110 | 10001 | 0 | — | R.S. |

| | | | | | |
|---|---|---|---|---|---|
| 3 | 00100 | 10001 | 0 | 10 | A - m |
| Shift | 00010 | 01000 | 1 | — | R.S |
| 4 | 11100 | 01000 | 1 | 01 | A + m |
| Shift | 11110 | 00100 | 0 | — | R.S |
| 5 | 11110 | 00100 | 0 | 00 | No. operation |
| Shift | 11111 | 00010 | 0 | — | RS. |

$AQ = 1111100010$

$Q_{15} = 1111100010_2 = -30_{10}$

$(-6) \times (+5) = -30$

Advantages :-

* It directly handles signed number
* It uses 2's complement represention
* It Reduces unnecessary addition operation
* A right shift presusses the sign
* It is efficient for signed multiplication

## 3) Booth's Multiplication Algorithm.

Q) Multiply $(-6) \times (+5)$

Given :-

$M = -6$

$Q = +5$

$-6 \times 5 = -30$

step 1 :- Represent -6

$+6 = 00110$

1's complement = 11001

$+1$

11010

step 2 :- Represent = +5

| | |
|---|---|
| 2 | 5 |
| 2 | 2 — 1 |
| | 1 — 0 |

$Q = 00101$

M = -6 = 11010

A = 00000

Q = 00101

M = 11010

Booth's Algorithm Rules:-

| $Q_0$ $Q_{-1}$ | operation |
|---|---|
| 0 0 | No operation |
| 0 1 | A = A+M |
| 1 0 | A = A-M |
| 1 1 | No operation |

After every operation, an arithmatic right shift is performed

4) Restoring and Non Restoring Division

13 ÷ 3

2 | 13
2 | 6 - 1
2 | 3 - 0
1 - 1

= (1101)$_2$

2 | 3
1

(0011)$_2$

Restoring Division

A = 00000

Q = 1101

M = 00011

Calculation :-

| step | operation | Result. |
|---|---|---|
| Initial | A=00000 , Q = 1101 | — |
| 1 | Shift & Subtraction | Negative -> Restore. |
| 2 | Shift & Subtraction | positive -> $Q_0$ = 1 |
| 3 | Shift & Subtraction | Negative - positive |
| 4 | Shift and Subtraction | Negative -> Restore |

A = 00001

Q = 0100

Q = 0100$_L$

P = 00001$_L$

B, Non Restoring Division.

In non-Restoring division, a Negative partial is not immediately Restored.

5, IEEE 754 Representation of -13.25

Step 1 :- Binary Conversion

13 -1101$_L$

0.25 = 0.01$_L$

-13.25 = -1101.01$_L$

-1101.01 = -110101 x $2^3$

S = 1, E = 3, F = 10101

Signal precision :-

Fraction :-

10101000 00000000 00000000

11000001 0101 00000000000000

Therefore

11000001010100000000000000

Hexadecimal

C1540000

Double Precision

Sign = 1 bit

Exponent = 11 bits

fraction = 52 bits

Bias 1023

$3 + 1023 = 1026$

$1026 = 10000000010_2$

1 | 10000000010 | 1010 | 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000 0000

Floating point Addition Analysis:-

- Compare Exponents
- Align mantissa
- add / subtract
- Normalize
- Panel
- Store

$-13.25 = -1101.01 \times 2^3$